

HÁZI FELADAT

Programozás alapjai 2.

Feladatválasztás/feladatspecifikáció

Kálmán Patrik

XG5YQ1

2025. április 5.

TARTALOM

1. Feladat	2
2. Feladatspecifikáció.....	2
3. Terv	3
3.1 Objektum terv.....	3
3.2 Algoritmusok.....	7

1. Feladat

Telefonkönyv!

Tervezze meg egy telefonkönyv alkalmazás egyszerűsített objektummodelljét, majd valósítsa azt meg! A telefonkönyvben az alábbi adatokat akarjuk tárolni:

- név (vezetéknév, keresztnév)
- becenév
- cím
- munkahelyi szám
- privát szám

Az alkalmazással minimum a következő műveleteket kívánjuk elvégezni:

- adatok felvétele
- listázás
- adatok törlése
- egyszerű keresés

A rendszer lehet bővebb funkcionálisú ezért nagyon fontos, hogy jól határozom meg az objektumokat. Demonstrálom a működést külön modulként tesztprogrammal! A megoldáshoz **nem** használok STL tárolót!

2. Feladatspecifikáció

A feladat egy egyszerű elektronikus telefonkönyv alkalmazás megtervezése és megvalósítása objektumorientált rendszer alapján. Mivel a telefonkönyvben tetszőleges számú bejegyzést kívánunk tárolni, az adatokat dinamikus memóriakezeléssel kezeljük.

Az adatok szöveg és szám formátumúak. Az alkalmazás lehetővé teszi a felhasználó számára, hogy korlátlan mennyiségű adatot rögzítsen.

Az alkalmazás a felhasználó számára az adatok felvételét, listázását, törlését, valamint a meglévő adatok keresését biztosítja. Az alkalmazás program megoldása további bővítésre is alkalmas, ezért a fejlesztés során kiemelt figyelmet fordítunk az objektumok megfelelő struktúrájára.

A program kezeli a hibákat. Amennyiben egy művelet sikertelen, a rendszer megfelelő visszajelzést ad a felhasználó számára.

A program tesztelésére egy külön modult készítünk, amely ellenőrzi a funkciók helyes működését a közölt adatokkal, valamint a tetszőleges a más által adott adatokkal is.

3. Terv

A feladat egy telefonkönyv rendszer megtervezését, valamint egy tesztprogram megtervezését igényli.

3.1 Objektum terv

A telefonkönyv alkalmazást több osztályból álló objektumrendszerrel valósítom meg. Az osztálystruktúra a következőképpen épül fel:

String osztály:

A String osztály egy egyszerű implementációja a C-sztringek (nullával lezárt karakterláncok) kezelésére. Az osztály támogatja a dinamikus memória kezelést, másoló konstruktort, operátorokat és egyéb fontos műveleteket.

Adattagjai:

- `pData`: pointer az adatra (`char*` típusú)
- `len`: hossz, lezáró nulla nélkül (`size_t` típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- `printDbg()`: Hibakeresési információ kiírása
- `size()`: A string hosszának lekérdezése
- **Operátor túlterhelések** a különböző műveletek elvégzéséhez
- `c_str()`: C sztringet ad vissza

Nev osztály:

A Nev osztály tárolja és kezeli egy személy nevét (vezetéknév, keresztnév, becenév). A név különböző formáit (pl. teljes név) visszaadhatjuk, és az osztály tartalmaz konstruktorokat, hogy a nevet különböző módokon inicializáljuk.

Adattagjai:

- `vnev`: Vezetéknév (`String` típusú)
- `knev`: Keresztnév (`String` típusú)
- `bnev`: Becenév (`String` típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (`getVnev()`, `getKnev()`, `getBnev()`)
- `getTeljesNev()`: A teljes név összeállítása
- Operátor túlterhelés a kiíratáshoz (`operator<<`)
- `Beolvas_nev()` metódus

Cim osztály:

A Cim osztály tárolja és kezeli egy cím adatait, beleértve az országot, irányítószámot, várost és utcát. Az osztály különböző konstruktorokat tartalmaz, és lehetővé teszi a címek különböző formájú beolvasását és kiírását.

Adattagjai:

- **ország:** Ország megnevezése (String típusú)
- **iranyitoszam:** Irányítószám (String típusú)
- **varos:** Város neve (String típusú)
- **utca:** Utca neve (String típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (`getOrszag()`, `getIrandito()`, `getVaros()`, `getUtca()`)
- Operátor túlterhelés a kiíratáshoz (`operator<<`)
- `Beolvas_cim()` metódus

Tel osztály:

A Tel osztály tárolja és kezeli egy személy telefonszámadatait, beleértve a munkahelyi és a privát telefonszámot. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve a telefonszámok kezelését, beolvasását és kiírását.

Adattagjai:

- **mszam:** Munkahelyi szám (String típusú)
- **pszam:** Privát szám (String típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (`getMszam()`, `getPszam()`,)
- Operátor túlterhelés a kiíratáshoz (`operator<<`)
- `Beolvas_tel()` metódus

Telefonkönyv osztály:

A Telefonkönyv osztály tárolja és kezeli a profilok listáját egy telefonkönyvben. Az osztály lehetővé teszi a profilok hozzáadását, törlését, keresését és listázását. Az adatok dinamikusan tárolódnak, és az osztály automatikusan kezeli a kapacitás növelését.

Adattagjai:

- **profilltar:** Profil típusú objektumokat tároló pointer
- **profilSzam:** A jelenleg tárolt profilok száma (size_t típusú)
- **kapacitas:** A telefonkönyv maximális kapacitása (size_t típusú)

Fontosabb metódusai:

- **Konstruktor** és **destruktor** a megfelelő memóriakezeléshez
- **addProfil():** Új profil hozzáadása a telefonkönyvhöz
- **torol():** Profil törlése megadott paraméter alapján
- **keres metódusok:** Különböző keresési lehetőségek (**név**, **cím**, **telefonszám** szerint)
- **noveloKapacitas():** A telefonkönyv kapacitásának dinamikus növelése
- **getProfilSzam():** Visszaadja a tárolt profilok számát

Profil osztály:

A Profil osztály tárolja és kezeli egy személy adatprofilját, beleértve a nevét, címét és telefonszámát. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve az adatok kezelését, beolvasását és kiírását.

Adattagjai:

- **nev:** Nev típusú objektum
- **cim:** Cim típusú objektum
- **tel:** Tel típusú objektum

Metódusai:

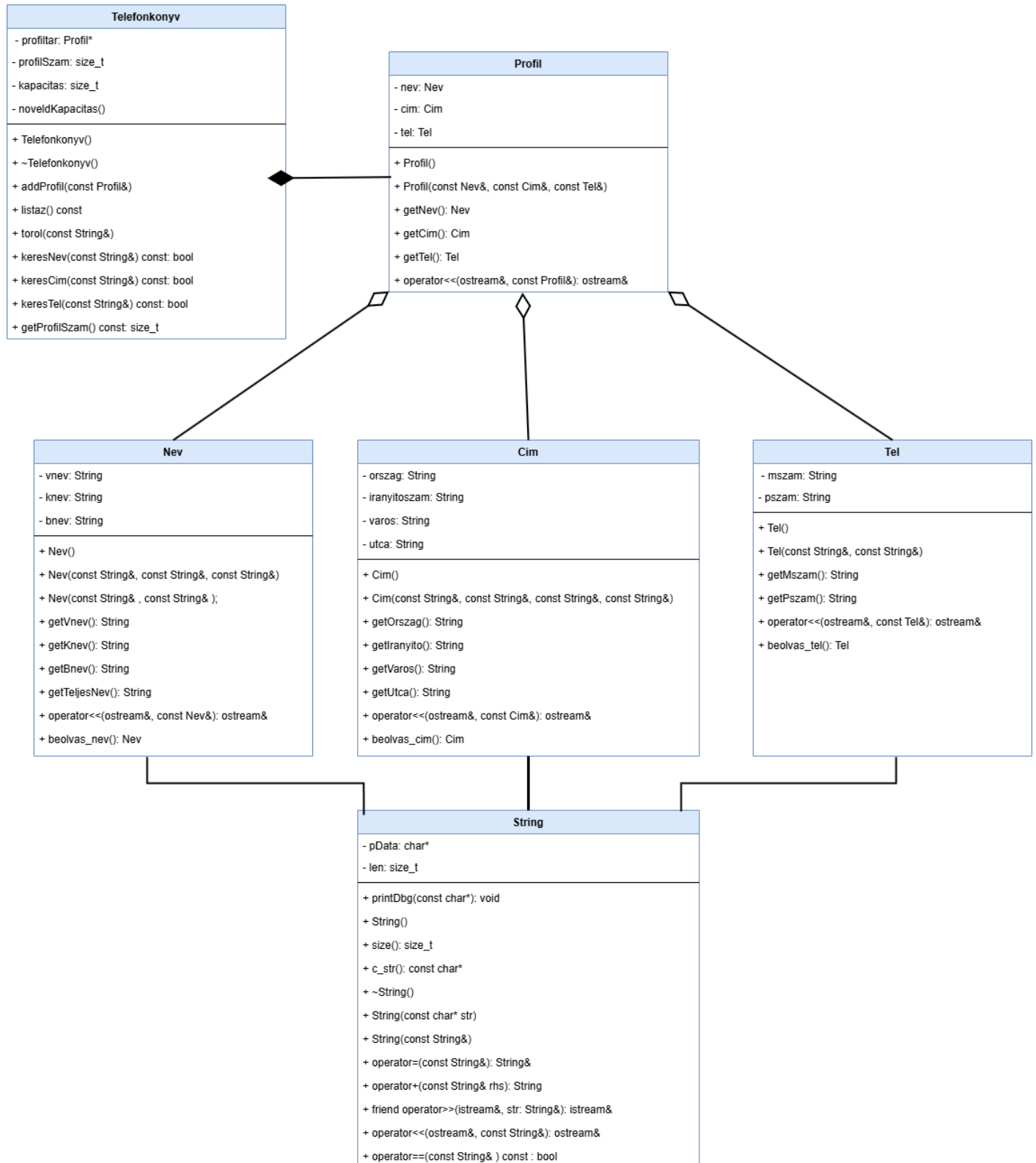
- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (**getNev()**, **getCim()**, **getTel()**)
- Operátor túlterhelés a kiíratáshoz (**operator<<**)

Az osztályok közötti kapcsolatok:

- A **Telefonkönyv** és **Profil** között **kompozíciós** kapcsolat van (kitöltött rombusz)
- A **Profil** és a **Nev**, **Cim**, **Tel** osztályok között **aggregációs** kapcsolat van (nem kitöltött rombusz)
- A **Nev**, **Cim**, **Tel** és **String** osztályok között **asszociációs** kapcsolat van (egyszerű vonal)
- Ez a struktúra lehetővé teszi a telefonkönyv rugalmas használatát, adatok hatékony kezelését és a bővíthetőséget is.

UML DIAGRAM:

Az osztálydiagramot a **draw.io** (diagrams.net) eszközzel készítettem, amely lehetővé tette számomra, hogy az UML jelölésekkel pontosan ábrázoljam az osztályok közötti kapcsolatokat és attribútumokat. Az elkészített diagramot exportáltam, és beillesztettem a dokumentumba, hogy bemutassam a telefonkönyv rendszer szerkezetét és működését.



3.2 Algoritmusok

A telefonkönyv optimális kapacitása, működés szempontjából „n” érték, amelyet a program figyelembe vesz, az n pontos értékét a tesztelés és az induló működés fogja meghatározni.

Profil hozzáadása

A telefonkönyvhez új profilt adunk hozzá, ha a kapacitás elérte a maximumot, növeljük a kapacitást, és másoljuk az adatokat.

Függvény addProfil(profil):

Ha profilSzam >= kapacitas akkor

Növeljük a kapacitást (noveldKapacitas())

Profilt adunk hozzá a profiltar tömbhöz az aktuális profilSzam indexre

Növeljük a profilSzamot

Kapacitás növelése

Ha a profilok száma meghaladja a tömb kapacitását, új tömböt hozunk létre, átmásoljuk az adatokat, majd töröljük a régi tömböt.

Függvény noveldKapacitas():

Növeljük a kapacitást 1-tel

Létrehozunk egy új Profil tömböt (ujTar) a növelt kapacitással

Átmásoljuk az adatokat a profiltárból ujTarba

Töröljük a régi profiltár tömböt

Beállítjuk profiltárat az ujTarra

Profil törlése név alapján

A keresett névvel rendelkező profilokat töröljük, majd az összes többi profilt eltoljuk, hogy a törölt helyet kitöltsük.

Függvény torol(vnev, knev):

Végig iterálunk a profiltar tömbön

Ha a profil neve megegyezik a keresett vnev és knev értékekkel:

Töröljük az aktuális profilt

Eltoljuk az összes következő profilt a tömbben

Csökkentsük a profilSzamot

Kiírunk egy üzenetet, hogy a törlés sikerült

Kilépés a ciklusból

Ha nem találtunk profilt:

Kiírunk egy üzenetet, hogy nem találtuk a profilt

Keresés név alapján

A keresett (név) alapján végig iterálunk a profilokon, és ha találunk egyezést, akkor kiírjuk a profil adatait.

Függvény keresNev(nev):

Beállítjuk a talalat változót hamisra

Végig iterálunk a profiltar tömbön

Ha a profil neve megegyezik a keresett névvel:

Kiírjuk a profilt

Beállítjuk a talalatot igazra

Ha nem találtunk találatot:

Kiírjuk az üzenetet, hogy nincs találat

Megjegyzés: ugyanez történik a cím és telefonszám alapján történő keresnél.

Név beolvasása

Lehetővé teszi a felhasználó számára, hogy megadja egy személy nevét, beleértve a vezetéknév, keresztnév és becenév mezőket, miközben biztosítjuk, hogy az adatok érvényesek.

Függvény beolvas_nev():

Kiírjuk a vezetéknévet

Beolvassuk a vezetéknévet

Ha vezetéknév érvényes:

Folytatjuk

Különben:

Kiírjuk hibát

Kiírjuk a keresztnévet

Beolvassuk a keresztnévet

Ha keresztnév érvényes:

Folytatjuk

Különben:

Kiírjuk hibát

Kiírjuk a becenevet

Beolvassuk becenevet

Visszatérünk a név objektummal

Megjegyzés: ugyanez történik a cím és telefonszám alapján történő beolvasásnál.